TANA
BUDDY

Product Vision, System Architecture, Implementation & Cost Guide

---

Prepared by: Favour Wanjui
Audience: CoAnthro Teammates — internal only
Version: 4.0 — Free-First Architecture Edition
Status: Final pre-build reference document

What's in this document?

Everything needed to build Tana Buddy — what it is, who it serves, how the data flows, how every component fits together, what each piece costs (most of it: nothing), and exactly what to build for the hackathon. Stack decisions have been made with cost as a hard constraint. Anything that has a free equivalent of equal quality uses the free option.

1. Background & Context

1.1 How Tana Works

Tana is a contracting agency and career launchpad for early career professionals in Kenya. It places people with client companies in Europe and actively supports them through that transition. The model runs in three stages:

1. **Recruitment —** Tana identifies and interviews early career talent in Kenya.
2. **Client Interview —** Shortlisted candidates interview with European client companies. If selected they are formally placed and contracted through Tana.
3. **The Programme —** The placed professional enters a structured 3-month programme. After 3 months, if things are going well, the contract extends for a full year. Tana Buddy stays active for as long as they are contracted through Tana.

1.2 Communication Infrastructure

Slack is the backbone of everything at Tana. Each client company has its own dedicated Slack channel where all professionals placed with that client post their EODs. The assigned Talent Success (TS) associate sees everything in that channel. Email exists for formal communication only.

Tana Buddy lives in Slack because that's where the work already happens. Asking people to go somewhere new is a UX tax we can't afford.

1.3 The Talent Success Team

Each professional has a dedicated TS associate. Currently one-to-many ratio. They read EODs, run manager sync calls, and intervene when something feels off. They are stretched. Anything that helps them stay on top of their portfolio without extra cognitive load is genuinely valuable.

2. Problem Statement

Early career Kenyan professionals placed with European companies face a high-stakes window where they must prove themselves to a foreign employer, adapt to unfamiliar norms, manage dual reporting, and grow fast — without the informal peer support or cultural context that would normally help them navigate it.

At the same time, TS associates are stretched across multiple professionals. They read EODs manually, follow up ad hoc, and often only find out someone is struggling after it's already become a serious problem.

> **The gap in one sentence:**
> The people doing the work have no intelligent guide. The people supporting them have no real-time visibility. Tana Buddy closes both gaps — inside Slack, where the work already lives.

3. What Is Tana Buddy?

A two-sided AI platform built specifically for the Tana placement programme. It lives in Slack and has a lightweight web dashboard for the TS team.

Side 1 — The Professional's Buddy	Side 2 — TS Associate Dashboard
Lives in Slack. Reads EODs, responds with insight, coaches via DM, tracks goals, monitors wellbeing.	Web dashboard. Portfolio health view, risk flags, manager sync prep, feedback logging.
Home: Slack — channel threads + DMs	**Home:** Next.js web app on Vercel
Frequency: Daily (3 months), biweekly after	**Frequency:** Weekly or when flagged

4. User Personas

4.1 The Professional

Persona	Amara, 24 — Junior Data Analyst
Situation	3 weeks into placement with a Dutch fintech. Remote from Nairobi. Managing a 2-hour timezone gap and a manager who communicates very differently from what she's used to.
Pain	Gets vague feedback and doesn't know how to act on it. Not sure if her EODs are landing. Feels she can't show vulnerability to her client manager.
Needs	A guide that feels safe, reads carefully, responds specifically, and helps her navigate this without making a big deal of how hard it actually is.
Quote	*"I work hard but I don't know if I'm growing in the right direction."*

4.2 The Talent Success Associate

Persona	Brian, TS Associate at Tana
Situation	Manages a portfolio of professionals across multiple client companies. Reads EODs in Slack, runs manager sync calls, intervenes when something feels off.
Pain	Too many messages to read deeply. Can't spot patterns without piecing it together manually. Goes into manager calls without a clear view of what to probe on.
Needs	A tool that reads everything and tells him what matters. Risk flags with context. Suggested questions before manager calls based on what's been showing up in a professional's recent EODs.
Quote	*"By the time I know someone is struggling, they're already halfway out the door."*

5. Feature Breakdown

5.1 Professional Side — Slack-First

EOD Reading & Response

- Thread reply in the channel: specific acknowledgement + one forward-looking prompt for tomorrow. Visible to the TS associate.
- DM to the professional: fuller response for a private deeper conversation without performing for the channel.

Conversational Coaching (DM)

The professional can DM Buddy at any time — unpacking feedback, prepping for a presentation, asking about workplace culture differences, or having a hard week. Warm first, practical second.

Goal Tracking

Goals set at placement start are stored and referenced naturally over time. Buddy surfaces drift without being mechanical.

Wellbeing Monitoring & Escalation

4. Buddy identifies concern signals across recent messages.
5. Assigned TS associate gets a private dashboard flag — a plain-language summary, not a transcript.
6. Flag includes a proposed response plan: suggested openers, what to listen for, call vs. message recommendation.
7. TS associate decides how to act. Buddy advises, doesn't override.

Post-3-Month Continuation

- Daily EODs replaced by biweekly reports — more reflective, less operational
- DM coaching stays available at any time
- Buddy stays active for as long as the professional is contracted through Tana

5.2 TS Associate Side — Web Dashboard

Portfolio Overview

- One row per professional: name, company, week number, health signal (Stable/Watch/At Risk), last EOD date, one-line AI summary, open flags

Individual Professional View

- Full timeline: EODs, AI summaries, goal progress, logged feedback, escalation history — searchable
- TS associate can ask Buddy directly: "What's been the main theme in Amara's EODs this month?"

Manager Sync Assistant

- Pre-call brief: what's going well, current concerns, 4-5 suggested questions for the manager
- Post-call log: TS associate records what they learned — feeds back into Buddy's context

Risk Flags

- Each flag: what triggered it, recent pattern, proposed response plan, resolution log

6. Data Architecture

6.1 Layer 1 — Live Professional Data

- EODs posted in the company's Slack channel
- Manager feedback relayed by the TS associate after sync calls
- Goals and milestone progress updates
- DM conversations with Buddy

> **Hackathon approach:**
> We create dummy professionals with 2 weeks of realistic EOD history pre-loaded. Architecture supports a clean swap to real data when Tana deploys.

6.2 Layer 2 — Tana's Institutional Knowledge

- Programme framework and 3-month curriculum
- Competency framework for evaluating progress
- Client company profiles — culture, norms, expectations
- Historical placement outcomes

> **Hackathon approach:**
> We build dummy versions of all of the above. Two fake client profiles, one fabricated competency framework, simulated historical outcomes. Real data drops straight in on deployment.

6.3 Layer 3 — External Reference Knowledge

Lives inside the AI model already. Referenced through prompt design, not built. European workplace culture, industry norms, professional development frameworks.

7. Full System Architecture & Implementation

This section covers the complete technical infrastructure. Every component, every connection, every workflow, the full database schema, the Claude prompt structure, and how every edge case is handled. This is what we build from.

7.1 Technology Stack — Free-First

Every decision below was made with cost as a hard constraint. If a free option exists that delivers the same value, we use it. Where cost is unavoidable, it's minimal and justified.

Layer	Tool	Cost	Why this choice
Slack Bot	Slack Bolt SDK (Node.js)	**FREE**	Official SDK. Handles events, DMs, slash commands. No cost to use the Slack API.
Frontend + API	Next.js on Vercel Hobby	**FREE**	Hobby plan covers unlimited deployments, serverless functions, and daily cron jobs. No server management.
Scheduling	Vercel Cron Jobs	**FREE**	Hobby plan allows daily cron triggers. One daily job checks what needs to happen today — covers all scheduled workflows without extra infrastructure.
Database	Xata	**FREE**	15GB free tier. PostgreSQL-compatible REST API. Team already has experience with it from the hackathon hub.
Rate limiting + dedup	Upstash Redis	**FREE**	10,000 commands/day on free tier. Used to prevent duplicate EOD processing and handle concurrent webhook events.
AI — EOD responses + coaching	Claude Haiku (claude-haiku-4-5)	**~$0.01–0.05/day**	Haiku is 20× cheaper than Sonnet and more than capable for EOD responses,

			coaching conversations, and health scoring. Reserved for all high-frequency calls.
AI — manager sync briefs	Claude Sonnet (claude-sonnet-4-5)	**~$0.05/call**	Sonnet only for manager pre-call briefs — these require deeper multi-document reasoning. Used sparingly, triggered manually.
Orchestration	Next.js API routes (no n8n)	**FREE**	n8n was removed. Its free cloud tier (2,500 executions/month) is exhausted in days at real usage. Self-hosting needs a server. Our workflows are simple enough to live in Next.js API routes.

Why n8n was dropped — and what replaced it

n8n Cloud's free tier caps at 2,500 workflow executions per month. With a daily engagement check running across even 15 professionals, that's gone in 6 days. Self-hosting n8n requires a server — that means Railway, Render, or Fly.io, all of which either cost money or spin down on inactivity (breaking webhook reliability).

The orchestration logic we actually need — receive Slack event, check conditions, call Claude, store in DB, post response — is straightforward enough to implement as a sequence of Next.js API route calls. No visual workflow builder required. No extra service to maintain.

What replaced each n8n responsibility:
Webhook receiving → Next.js API route (/api/slack/events)
Scheduled jobs → Vercel Cron (free, daily trigger)
Retry logic → try/catch with exponential backoff in the API route
Condition branching → plain if/else in the handler function
Rate limiting / deduplication → Upstash Redis (free tier)

Total estimated cost for the hackathon demo

Claude API (Haiku): 3 professionals × 14 days × 1 EOD = 42 calls. ~3,500 input tokens + 500 output tokens each. Total: ~$0.10
Claude API (Sonnet): ~5 manager sync brief generations during testing. Total: ~$0.25
Everything else: Slack, Vercel, Xata, Upstash Redis — all free tier.

Hackathon total: under $0.50
Production estimate (50 professionals, monthly): Haiku for 50 × 30 EODs = 1,500 calls ≈ $3.80. Sonnet for ~20 sync briefs ≈ $1.00. All infrastructure still free. Total: ~$5/month.

7.2 How Everything Connects

This is the full picture. Read this before looking at any individual workflow.

Architecture Overview (Free-First Stack)

SLACK WORKSPACE

```
[Company Channel] ─── professional posts 'EOD: ...'
[DM with Buddy]   ─── coaching conversation
        │
        ▼  Slack Events API → POST to Next.js /api/slack/events
```

NEXT.JS BACKEND (Vercel)

```
/api/slack/events
  ├── EOD detected?
  │     ├── Check Upstash Redis: already processed this msg_id? → skip
  │     ├── Fetch professional profile + last 10 EODs from Xata
  │     ├── Call Claude Haiku API → get JSON response
  │     ├── Store EOD + signals in Xata
  │     ├── Post thread reply to Slack channel
  │     ├── Send DM to professional
  │     └── wellbeing_concern? → create flag in Xata + DM TS associate
  └── DM message? → coaching flow → Claude Haiku → DM response
```

VERCEL CRON (daily, 06:00 UTC = 09:00 EAT)

```
/api/cron/daily-check
  ├── For each active professional: last EOD > 2 days ago? → send gentle DM
  ├── Last EOD > 4 days? → create engagement flag → DM TS associate
  ├── It's Friday? → send reflection prompt DM to all professionals
  └── Professional past 3 months + biweekly due? → send report reminder DM
```

TS DASHBOARD (Next.js, same Vercel project)

```
Browser ◀── polls /api/dashboard/* ◀── Xata DB
'Generate sync brief' → POST /api/sync-brief → Claude Sonnet → return to UI
TS logs feedback → POST /api/manager-feedback → Xata
```

7.3 The Workflows — Step by Step

There are 5 distinct workflows. All implemented as Next.js API routes and one Vercel Cron endpoint. No external orchestration service needed.

Workflow 1: EOD Ingestion — /api/slack/events (POST)

Triggered by: Slack Events API webhook on every message.channels event

Step 1 — Verify: Validate the Slack signing secret. Return 200 immediately to avoid Slack retries (Slack times out at 3s). Offload processing to a background async function.

Step 2 — Deduplicate: Check Upstash Redis for key eod:{message_ts}. If exists, stop — already processed. If not, set the key with 24hr TTL.

Step 3 — Detect: Does the message text start with 'EOD:' or '/eod'? Is it longer than 50 characters? Is the channel ID in our list of company channels? If any check fails, stop.

Step 4 — Fetch context: Query Xata: get professional by slack_user_id, client company profile, goals, and last 10 EODs.

Step 5 — Build prompt & call Claude Haiku: Assemble the system prompt (see Section 7.5). Call the Anthropic API. Parse the JSON response.

Step 6 — Store: Write the EOD record to Xata (content, thread response, DM response, health signal, reasoning, wellbeing_flagged). Update professional's current_health and last_eod_at.

Step 7 — Respond in Slack: Post the thread_response as a reply to the message. Send the dm_response as a DM to the professional.

Step 8 — Escalate if needed: If wellbeing_concern is true: create a flag record in Xata. DM the professional's TS associate with a brief notification pointing them to the dashboard.

Workflow 2: DM Coaching — /api/slack/events (POST, message.im events)

Triggered by: any DM the professional sends to the Buddy Slack app

Step 1: Verify Slack signing secret. Deduplicate via Redis.

Step 2: Identify professional from Slack user ID. Fetch their profile and last 10 conversation turns from the conversations table.

Step 3: Build a lighter prompt — coaching context, not EOD analysis. Call Claude Haiku.

Step 4: Store both turns (user message and assistant response) in the conversations table. Send the response as a DM reply.

Step 5 (optional): If Claude's response signals a potential wellbeing concern in a DM conversation, trigger the same escalation flow as Workflow 1 Step 8.

Workflow 3: Daily Cron — /api/cron/daily-check (GET, Vercel Cron)

Triggered by: Vercel Cron at 06:00 UTC (09:00 AM EAT) every day

Step 1: Authenticate the cron request (Vercel passes a secret header — verify it).

Step 2 — Engagement check: Fetch all active professionals. For each: how many business days since their last EOD? 2 days → send a gentle DM check-in. 4+ days → create an 'engagement' flag in Xata and DM the TS associate.

Step 3 — Friday prompt: If today is Friday: generate a brief personalised reflection prompt per professional using their recent EOD themes (one Claude Haiku call per professional). Send as DM.

Step 4 — Biweekly reminders: For professionals past the 3-month mark: calculate days since their last EOD/biweekly report. If 14+ days: send a biweekly report reminder DM.

Step 5 — Stale manager feedback alert: For each professional: days since last manager_feedback record? If 14+ days: add a soft reminder in the TS dashboard (no DM, just a dashboard indicator).

Workflow 4: Manager Sync Brief — /api/sync-brief (POST)

Triggered by: TS associate clicking 'Generate pre-call brief' in the dashboard

Step 1: Receive professional_id from the dashboard request. Authenticate via dashboard session.

Step 2: Fetch: last 14 days of EODs, health score history, open flags, goals, and any manager feedback on record.

Step 3: Call Claude Sonnet (not Haiku — this needs deeper multi-document reasoning). Prompt: given this data, produce a structured pre-call brief with: what's going well, current concerns, and 4-5 specific questions to ask the manager.

Step 4: Store the brief in Xata's sync_briefs table. Return it to the dashboard immediately.

Workflow 5: Biweekly Report Processing — handled by Workflow 1 (same code path)

Triggered by: professional responds to the biweekly reminder DM with their report

How it works: When the professional DMs their biweekly report, Workflow 2 (DM coaching) picks it up. The message is longer and more reflective — the prompt detects this and shifts to biweekly analysis mode rather than daily coaching mode. The same EOD ingestion logic applies: store, score, respond, flag if needed.

What changes: The report_type field in the eods table is set to 'biweekly'. The health score update uses a lower signal-frequency expectation. The Claude prompt is tuned for reflective depth rather than daily check-in brevity.

7.4 Database Schema — Xata (Free Tier)

All data lives in Xata. Free tier gives 15GB — more than enough. Here are every table and column, with notes on why each exists.

Table: professionals

```
id                  string   (Xata auto-generated)
name                string
slack_user_id       string   -- matches Slack event user IDs to profiles
email               string
client_company_id   string   -- FK → client_companies
ts_associate_id     string   -- FK → ts_associates
start_date          datetime
phase               enum: '3-month' | '1-year'
status              enum: 'active' | 'extended' | 'exited'
goals               JSON     -- [{id, title, description, target_date,
status}]
current_health      enum: 'stable' | 'watch' | 'at_risk'
last_eod_at         datetime -- used by daily cron for engagement checks
```

Table: client_companies

```
id                  string
name                string
country             string
slack_channel_id    string   -- the Slack channel we listen to for EODs
culture_notes       text     -- loaded from dummy/real knowledge base
typical_expectations text
```

```
manager_name        string
manager_email       string   -- for reference only, manager not in system
```

Table: ts_associates

```
id                  string
name                string
slack_user_id       string   -- for DM escalation notifications
email               string
```

Table: eods

```
id                  string
professional_id     string   -- FK → professionals
content             text     -- raw EOD text
slack_message_ts    string   -- Slack timestamp, used as thread reply target
slack_channel_id    string
posted_at           datetime
thread_response     text     -- what Buddy posted in the channel thread
dm_response         text     -- what Buddy sent via DM
health_signal       enum: 'stable' | 'watch' | 'at_risk'
health_reasoning    text     -- AI's plain-language explanation
wellbeing_flagged   boolean  -- whether this EOD triggered an escalation
processed_at        datetime
report_type         enum: 'eod' | 'biweekly'
```

Table: conversations

```
id                  string
professional_id     string   -- FK → professionals
role                enum: 'user' | 'assistant'
content             text
created_at          datetime
context_type        enum: 'dm_coaching' | 'biweekly'
```

Table: flags

```
id                  string
professional_id     string   -- FK → professionals
type                enum: 'wellbeing' | 'engagement' | 'performance'
summary             text     -- AI-generated, shown to TS associate
proposed_plan       text     -- AI-generated suggested response
triggered_at        datetime
ts_notified_at      datetime
resolved_at         datetime -- null if still open
```

```
resolution_notes    text     -- what the TS associate did
dismissed           boolean  -- TS associate marked as false positive
```

Table: manager_feedback

```
id                  string
professional_id     string   -- FK → professionals
ts_associate_id     string   -- FK → ts_associates (who logged it)
content             text     -- what the manager said, paraphrased by TS
call_date           datetime
logged_at           datetime
```

Table: sync_briefs

```
id                  string
professional_id     string   -- FK → professionals
content             text     -- AI-generated pre-call brief
generated_at        datetime
used_at             datetime -- null until TS associate marks it used
```

Table: settings

```
key                 string   -- e.g. 'cron_last_run', 'onboarding_version'
value               string
```

7.5 The Claude Prompt Architecture

The prompt is what makes Buddy feel real rather than robotic. Getting this right is the highest-leverage technical work in the project. Here's the full structure.

Model Selection by Call Type

Call type	Model	Why	Approx cost per call
EOD response	`claude-haiku-4-5`	Volume is high — runs on every EOD	~$0.003
DM coaching conversation	`claude-haiku-4-5`	Conversational, doesn't need deep reasoning	~$0.001

Health score recalculation (weekly)	`claude-haiku-4-5`	Pattern detection, high frequency	~$0.004
Manager sync brief	`claude-sonnet-4-5`	Needs deep reasoning across 14 days of data	~$0.05
Friday reflection prompt generation	`claude-haiku-4-5`	Short personalised prompt — Haiku handles this	~$0.001

EOD System Prompt Structure

This is loaded fresh for every EOD call. It tells Claude who it is, who it's talking about, and everything it needs to know.

```
You are Tana Buddy — an AI companion built for Tana, a Kenyan contracting
agency that places early career professionals with European client companies.

You are supporting: {professional.name}, a {professional.role} placed with
{company.name} in {company.country}. They are in week {week_number} of
their placement.

About {company.name}:
{company.culture_notes}
{company.typical_expectations}

This professional's current goals:
{goals list}

Their recent EOD history (oldest to newest, last 10):
{eod summaries with dates}

Current health signal: {current_health}
Open flags: {active flags or 'none'}

What week {week_number} typically looks like in the Tana programme:
{tana_framework_section_for_this_week}

Read their EOD carefully. Respond as someone who genuinely knows them,
cares about their success, and actually thinks about what they've written.
Not a manager. Not a chatbot. A warm, honest, practically useful guide.

Respond ONLY with valid JSON matching this exact schema:
```

Required JSON Response Schema

```
{
  "thread_response": "string — 2-3 sentences, warm and specific, for the
channel thread",
  "dm_response": "string — fuller private response. Can include a follow-up
question.",
  "health_signal": "stable | watch | at_risk",
  "health_reasoning": "1-2 sentences explaining the signal assigned",
  "wellbeing_concern": false,
  "wellbeing_summary": null,
  "proposed_plan": null,
  "goal_updates": []
}

When wellbeing_concern is true:
  wellbeing_summary: plain-language description — NO verbatim quotes from the
professional
  proposed_plan: specific actionable suggestions for the TS associate
  goal_updates: [{goal_id, note}] if any goals seem affected
```

Context Window Management

Context has a cost ceiling. We manage it by including the last 10 EODs as lightly compressed summaries rather than full verbatim text. For professionals past their first month, older EODs are progressively summarised into a one-paragraph rolling context block. Full history stays in Xata — only what fits meaningfully in the context window goes to the API.

7.6 Slack App Configuration

Everything the Slack app needs to be set up correctly. Do this first — everything else depends on it.

Required OAuth Scopes

Scope	What it enables
`channels:history`	Read messages in channels the app is added to — needed to receive EODs
`channels:read`	List channels — verify company channel IDs on setup
`chat:write`	Post messages — thread replies and channel notifications

`im:write`	Send DMs to users — coaching conversations and escalation alerts
`im:read`	Read DMs — for the coaching conversation flow
`im:history`	Read DM history — maintain conversation context across sessions
`users:read`	Look up user info — match Slack user ID to a professional profile
`users:read.email`	Match by email as fallback when user ID lookup fails

Event Subscriptions (Request URL: your Vercel deploy URL + /api/slack/events)

- message.channels — fires on every channel message. How we catch EODs.
- message.im — fires when someone DMs the app. How we receive coaching messages.

EOD Detection

Professionals start their EOD message with 'EOD:' or use the /eod slash command. The API route checks for this prefix in step 3 of Workflow 1. Simple, reliable, easy to explain. A classifier can replace this later if Tana wants to drop the prefix requirement — but keyword detection is the right call for the hackathon.

7.7 Environment Variables

Every secret lives here. Nothing hardcoded. Add these to Vercel's environment settings.

```
# Slack
SLACK_BOT_TOKEN=xoxb-...          # Bot User OAuth Token
SLACK_SIGNING_SECRET=...           # From Slack app settings → Basic
Information
SLACK_APP_TOKEN=xapp-...           # Only needed for Socket Mode (local dev)

# Anthropic
ANTHROPIC_API_KEY=sk-ant-...

# Database — Xata
XATA_API_KEY=xau_...
XATA_DB_URL=https://workspace.region.xata.sh/db/tana-buddy:main

# Upstash Redis — for deduplication and rate limiting
UPSTASH_REDIS_REST_URL=https://...upstash.io
UPSTASH_REDIS_REST_TOKEN=...
```

```
# Auth — dashboard password protection
DASHBOARD_PASSWORD=...
AUTH_SECRET=...                    # 32+ random characters for cookie signing

# Vercel Cron auth
CRON_SECRET=...                    # Random string — Vercel passes this in cron
requests
```

Vercel Cron Configuration (vercel.json)

```
{
  "crons": [
    {
      "path": "/api/cron/daily-check",
      "schedule": "0 6 * * *"
    }
  ]
}
```

7.8 Edge Cases & Failure Handling

These are the scenarios that will happen in production. Each one has a clear, implemented response.

Scenario	What happens	How it's handled
Slack sends the webhook but our endpoint takes > 3 seconds	Slack marks delivery failed and retries	We return HTTP 200 immediately and process asynchronously. The deduplication Redis key prevents the retry from being processed twice.
Claude API times out or returns an error	No response appears for the professional	Try/catch with 2 retries (1s, then 3s backoff). On third failure: post a fallback thread reply ('Got your EOD — I'll have a response shortly') and log the error. TS associate sees no health update, which itself is a soft signal.
Claude returns malformed JSON	JSON.parse throws	Catch the error, retry once with a simpler prompt that explicitly says 'respond only with valid JSON'. If still malformed: send a generic warm acknowledgement and log for review.

Non-EOD message posted in company channel	Workflow 1 receives the event	Step 3 checks for 'EOD:' prefix. If absent, the function returns early. No processing, no response, no cost.
EOD is fewer than 50 characters	Too little signal for meaningful AI analysis	Reply in thread: 'That's a short one — anything you want to add? Even a sentence or two about how today actually went.' Don't run the full processing. Store a stub EOD record.
Professional posts two EODs in one day	Both trigger Workflow 1	Redis dedup key uses message_ts (unique per message), so both are processed. The second is stored with a duplicate_day flag in Xata. Health scoring uses only the most recent.
Wellbeing flag is a false positive	TS associate acted on something that wasn't a real problem	TS associate marks the flag as 'dismissed' in the dashboard. The dismissal is logged. Over time, dismissed flags inform future prompt tuning.
Professional goes silent for 4+ business days	Daily cron detects no recent EOD	Day 2: gentle DM to professional. Day 4+: create an 'engagement' flag in Xata and DM the TS associate. The flag includes the last known context so the TS associate has something to work with.
Upstash Redis is unavailable	Deduplication check fails	Fail open: log the Redis error but continue processing. In the worst case, a duplicate response is sent once. This is better than dropping EODs silently. Upstash's free tier uptime SLA is 99.9%.
Professional exits Tana	Buddy should stop responding	TS associate sets status = 'exited' in the dashboard. Workflow 1 checks status at Step 4 — if 'exited', stops immediately. Cron skips exited professionals entirely.
New professional joins with no EOD history	Empty context passed to Claude	The prompt acknowledges this: 'This is [Name]'s first EOD. No prior history available. Respond with extra warmth and orient around their goals and first impressions.' After 3 EODs the full context window is in play.

TS associate hasn't logged manager feedback in 2+ weeks	Manager-side data goes stale	Daily cron checks last manager_feedback date per professional. If 14+ days: a soft indicator appears in the dashboard on the professional's row. No DM — just a visible nudge in the UI.
Vercel Cron doesn't fire (platform issue)	Engagement checks and reminders are missed for a day	Cron runs daily so a missed day means at most a 24-hour delay. The daily check is idempotent — when it next runs it catches up on anyone who should have been flagged the day before.

7.9 End-to-End Flow Walkthrough

The complete sequence from Amara posting her EOD to Brian seeing the updated health score. Every system, in order.

8. Amara posts 'EOD: Today I finished the dashboard component...' in #dutch-fintech-co at 6:47 PM EAT.
9. Slack detects a message.channels event. Sends a POST to /api/slack/events on our Vercel deployment within milliseconds.
10. Our handler returns HTTP 200 immediately. Starts async processing in the background.
11. Redis check: has eod:{message_ts} been processed? No. Set the key with 24hr TTL.
12. EOD detection: starts with 'EOD:', longer than 50 characters, channel_id matches dutch-fintech-co. All checks pass.
13. Xata query: fetch Amara's profile, goals, and last 10 EODs. Fetch dutch-fintech-co company profile.
14. Build Claude Haiku prompt with all context. Call Anthropic API. Wait ~2-4 seconds.
15. Claude returns valid JSON: thread_response, dm_response, health_signal: 'watch', health_reasoning, wellbeing_concern: false.
16. Xata write: store EOD record with all fields. Update Amara's current_health to 'watch', last_eod_at to now.
17. Slack API: post thread_response as a reply to Amara's message in #dutch-fintech-co.
18. Slack API: open DM with Amara, send dm_response.
19. wellbeing_concern is false. No escalation needed. Workflow ends.
20. Total elapsed time: approximately 6-10 seconds from Amara hitting send.
21. Next morning: Brian opens the dashboard. Amara's row shows 'Watch'. He clicks in, reads the health reasoning, and decides to check in with her later in the week.

7.10 Hackathon Build Sequence

The order that makes sense given the timeline and dependencies. Each step assumes the previous one is working before moving on.

#	Task	Notes
1	**Create the dummy data set — 2 client companies, 3 professionals, 14 days of EODs each**	Do this before any code. The prompt only impresses judges if it has realistic context to work with.
2	**Set up Xata DB — create all tables from Section 7.4**	Use the Xata web dashboard. No migrations needed — just create tables and columns manually.
3	**Set up Upstash Redis — create a free database**	Takes 2 minutes. Copy the REST URL and token into env vars.
4	**Create the Slack app in a test workspace — configure scopes and event subscription URL**	Point the event URL at your local dev tunnel (ngrok or Vercel preview URL) initially.
5	**Build /api/slack/events — Steps 1-3 of Workflow 1 (verify, deduplicate, detect)**	Get the webhook receiving and EOD detection working before touching Claude.
6	**Build and test the Claude Haiku prompt — iterate until responses feel genuinely good**	This is the highest-leverage step. Don't rush it. Test against all 3 dummy professionals.
7	**Complete Workflow 1 — Steps 4-8 (fetch, prompt, store, post thread, DM, escalate)**	Wire everything together. One full EOD-to-response flow end to end.
8	**Build the TS dashboard — portfolio view first, then individual professional view**	Use polling (5s) for real-time updates. Start with the portfolio view — it's what judges see first.
9	**Build /api/sync-brief + the 'Generate pre-call brief' button in the dashboard**	This is the demo's most impressive moment. Make the brief genuinely useful.
10	**Set up Vercel Cron + /api/cron/daily-check — basic version (engagement check only)**	Don't need all 5 cron steps for the demo. Engagement check is enough to demonstrate the concept.
11	**Pre-load one wellbeing flag scenario in the dummy data**	Set it up so you can show the escalation flow live. Don't rely on triggering a real one during the demo.

12	**Run the full demo flow as a team — find every rough edge before the judges do**	Do this at least twice. Time it. The 15 minutes goes faster than you think.

8. Trust, Privacy & The Data Boundary

Tana Buddy's value depends entirely on professionals being honest with it. Honesty only happens if they genuinely trust it. Trust is built through explicit clarity.

8.1 The Policy — Plain Language

What Tana Buddy does with your messages

Your EODs: Buddy reads and stores your EOD. Your TS associate can always read your EODs in Slack — that hasn't changed. Buddy adds a layer of analysis that helps your associate support you better.

Your DMs with Buddy: Private by default. Your TS associate does not see the raw content of your DMs. What they may see is a high-level signal if Buddy detects consistent signs of serious distress.

Your goals: Visible to your TS associate. Progress notes stored and summarised so they have a clear picture.

If Buddy detects a serious concern: Your TS associate is privately notified — a summary and a suggested response plan, not a transcript. Someone should be looking out for you.

8.2 The Data Boundary

TS associate CAN see	Stays private to you
All EODs — always visible in Slack	Raw DM content with Buddy
Goals and stated progress	Draft messages not sent
Health signal: Stable / Watch / At Risk	Exact words in any flagged message
Wellbeing flags — summary only, no verbatim	The fact that a flag was triggered (unless they bring it up)
Manager feedback logged by TS associate	Anything marked personal in a reflection

8.3 The Manager Boundary

Client managers are not on the platform. They have no access to anything. Manager input reaches the system only through the TS associate logging feedback after sync calls. The professional needs to know Buddy is on their side — not reporting to their manager.

9. Gaps, Assumptions & Open Questions

9.1 Gaps

Gap 1: No access to Tana's real data

Tana has the institutional knowledge. We don't have access to it for the hackathon. This is a resource constraint, not a product design gap.

How we handle it: Build dummy versions of everything. Architecture supports a clean swap to real data on deployment.

Gap 2: Unknown state of historical placement records at Tana

Historical data on what struggling professionals looked like before they exited would make risk detection smarter.

How we handle it: We assume records exist and simulate plausible ones. Even rough patterns recalled by TS associates are valuable when Tana eventually provides them.

Gap 3: Adoption is not guaranteed

A great tool fails if professionals treat it as a formal reporting channel instead of a safe space.

What Tana should do: Introduce Buddy alongside the TS associate in week 1, not instead of them. Make the DM coaching useful in the first 3 interactions. Build a 30-day check-in into the TS workflow where they ask the professional how Buddy has been. Make the trust policy genuinely clear at onboarding.

Gap 4: No direct manager data channel

Manager feedback reaches the system only when the TS associate logs it. The quality of that data depends on the TS associate's habit.

How we handle it: The sync brief feature (Workflow 4) makes the TS associate's calls more targeted, which improves what comes back. Stale feedback indicators in the dashboard nudge the habit without being annoying.

9.2 Assumptions

- **Access:** Professionals have reliable enough Slack access to post EODs regularly. WhatsApp fallback is a future consideration if connectivity is inconsistent.
- **Slack permissions:** Tana's Slack workspace allows installing a custom app with the required scopes. Confirm this before build day.
- **Team discipline:** TS associates log manager feedback consistently. This needs to be a formal part of their workflow from day one, not an optional extra.

- **AI quality floor:** Claude Haiku quality is sufficient for the responses we need. Test this early — if edge cases expose significant limitations, specific call types can be upgraded to Sonnet.

9.3 Open Questions

Question	Why it matters	Owner
Does Buddy respond to every EOD or only ones past a minimum-quality threshold?	Too many generic responses = noise	**Product**
What's the minimum dummy dataset that makes the demo impressive?	Determines pre-build data work	**All**
Biweekly reports — free text DM or a structured prompt conversation?	Affects data quality and UX	**Product**
How does Buddy handle someone going quiet for 4+ days if they're in a different timezone?	Cron fires at 9am EAT — may need timezone awareness	**Tech**

10. Hackathon MVP Scope

We're building both sides. Priority: each part must feel genuinely real. A Slack bot that reads one EOD beautifully beats a half-working bot that reads ten EODs poorly.

10.1 Building

#	What	Why it makes the cut
1	**Dummy data — 2 companies, 3 professionals, 14 days of EODs**	Build this first. Everything else depends on it.
2	**Slack bot — EOD detection, thread reply, DM response**	The core behaviour. If this works well, the demo works.
3	**Claude Haiku integration with structured JSON output**	The AI layer is the product. Get this right before building UI.
4	**Xata DB with all tables from Section 7.4**	Everything depends on reliable data storage.
5	**Upstash Redis for deduplication**	Prevents duplicate processing. 10-minute setup.
6	**DM coaching flow (Workflow 2)**	Shows the private side of Buddy. Critical for the trust story.
7	**TS dashboard — portfolio view + individual professional view**	Shows the other side of the platform. Makes the full picture visible.
8	**Manager sync brief — Workflow 4 + dashboard button**	Most impressive demo moment. Judges will remember this.
9	**One pre-canned wellbeing escalation scenario**	Judges will ask about edge cases. Show them one working cleanly.
10	**Vercel Cron — basic daily engagement check**	Demonstrates the automated care loop. Even if not live during demo, show the code.

10.2 Not Building for the Demo

- Friday reflection prompts and biweekly report reminders — describe in presentation

- Full onboarding consent flow — show the policy document, explain verbally
- Cohort analytics — one client, a few professionals is enough
- Stale manager feedback dashboard indicator — mention it, don't build it

11. What Success Looks Like

For the Professional

- They post an EOD and get a thread reply that reads like someone paid careful attention
- The DM response is warm, specific, and actually useful
- They feel like Buddy is on their side

For the Talent Success Associate

- The dashboard shows who needs attention without reading every message
- The pre-call brief saves prep time and makes the manager sync more targeted
- A risk flag feels credible and actionable

For Tana Long-Term

- Placement completion rate improves — fewer early exits
- Client satisfaction holds or improves
- Tana has a documented, data-backed picture of what successful placement looks like — valuable in client conversations and fundraising

12. Immediate Next Steps

#	Action	Output	Owner
1	Build dummy data — 2 client company profiles, 3 professional personas, 14 days of realistic EODs per person. Make them vivid enough that the AI's responses are genuinely impressive.	Dummy data folder ready to load	**All hands**
2	Set up Xata DB tables from Section 7.4 and seed with dummy data.	DB live with data	**Tech**
3	Set up Upstash Redis free account. Copy credentials to env vars.	Redis live	**Tech**
4	Create the Slack app. Configure all scopes from Section 7.6. Point event URL to local dev.	Bot receiving events	**Tech**
5	Build and iterate on the Claude Haiku prompt against dummy EODs. Don't move on until responses feel genuinely good.	Approved prompt v1	**Tech + Product**
6	Build /api/slack/events — Workflow 1 end to end.	Full EOD → response loop working	**Tech**
7	Wireframe the TS dashboard portfolio and individual views.	Approved wireframes	**Design**
8	Write the trust policy as a one-page plain-language document.	Policy doc for judges	**Product**

Free where possible. Cheap where not. Legit throughout.

The stack is decided. The schema is ready. The workflows are mapped. Now we build.

Let's go.

— Favour